[박종혁] Backend Engineer

devtaco30@gmail.com

운영을 통해 설계를 배웠습니다.

실시간 가상자산 지수와 뉴스 커뮤니티 플랫폼을 설계·운영하며 지난 5년간 백엔드를 담당했습니다. 최근 1년은 AI 부트캠프를 마치고 AI 면접 멘토 서비스와 운동 앱을 출시·운영하며 기획부터 배포 이후 운영까지 서비스 전반을 경험했습니다.
경력 초기에는 눈앞의 문제를 빠르게 해결하는 데 집중했습니다. 하지만 여러 시스템을 운영해 보면서 생각이 달라졌습니다. 장애가 생기면 바로 고치기 전에, 현재 상황을 파악해 가능한 대안을 비교하는 일이 더 중요했습니다. 이후에는 대안마다 트레이드오프를 확인한 뒤 방향을 정하고, 그 근거를 ADR로 남기고 있습니다.

기술 스택

- **주력** : Java · Kotlin · Spring Boot · JPA · MSA · Event-Driven
- **활용** : Python · TypeScript · Swift · React Native
- **AI** : Claude API · Cursor · Claude Code
- **Infra/DB** : AWS, Docker, Jenkins, PostgreSQL, MySQL, MongoDB, Redis, OpenSearch

경력 요약

(주)블루밍비트 | 2023.08 - 2025.08 | 개발팀 / 팀원

가상 자산 뉴스 커뮤니티 플랫폼 운영 및 성능 안정화

- **이벤트 기반 구조 전환** : 후속 작업 실패가 뉴스 발행 전체에 영향을 주는 구조를 메시지 큐로 격리
- **계약사 뉴스 제공 MSA** : 뉴스 발행 이벤트를 수신해 계약사별 포맷으로 변환·전송하는 독립 파이프라인 구축
- **검색 응답 지연 단축** : 파편화된 검색 구현을 일원화하고 병렬 조회로 전환 — **1.5~3초 → 200~450ms**
- **알림 발송 구조 전환** : 탐지 불가한 외부 SaaS 결함을 FCM 전환과 알림 설정 판정 내재화로 해소
- **블루밍비트 2.0 런칭** : 배포 환경 부재 상태에서 AWS 인프라 구축 (EB 기반) 및 Jenkins CI/CD 구성
- **Backend 리빌딩** : 일관성 없는 레거시 구조를 표준화하여 기술 부채 해소 및 유지보수 비용 절감
- **가상자산 표준 코드 관리 MSA** : 거래소별 티커 혼재로 인한 데이터 불일치를 표준 코드 체계로 해소
- **정보 알림봇 MSA** : 협력사 제공 차트 변동 이벤트를 수신, 뉴스 발행 및 알림 발송하는 파이프라인 구축
- **AppPush 시스템 개선** : 알림 유형 확장에 따른 발송 전략 재설계 및 묶음 발송 도입
- **AI 센티멘트 분석 리포트** : ChatGPT API로 뉴스 요약 기반 시황 리포트를 생성·적재

(주)웨이브릿지 | 2020.06 - 2023.08 | IT 본부 Infra 팀 / 팀원

가상 자산 실시간 데이터 수집 및 인덱스(지수) 산출 관리

- **분산 스케줄링 설계** : Job 단위 분리로 단일 서버 한계 극복 및 HA 환경 확보
- **실시간 지수 정합성 확보** : 데이터 경합 원인 추론 및 구조적 격리로 지수 이상치 제거
- **실시간 지수 산출 시스템 구축** : 백엔드·인프라·프론트 전 영역 담당하며 서비스 출시 주도
- **외부 고객사 데이터 제공** : 산출한 지수를 API·소켓으로 전달하고, 고객사별 지수 권한을 키로 검사
- **마켓정보 수집 및 표준 코드 관리** : 실시간 시장 데이터 수집 및 거래소별 티커 표준 코드 체계 관리

Side Builds

AI 도구를 활용해 기획부터 구현까지 진행한 개인 프로젝트입니다.

일부 프로젝트는 1인 개발사 Habitworks를 통해 출시·사업화까지 진행했습니다.
1인 개발 환경에서도 설계 결정의 맥락을 남기기 위해, 주요 선택지와 트레이드오프는 ADR로 기록하고 있습니다.

Apex — AI 면접 멘토

개요 : 개발자 면접 준비 과정에서 스스로 약점을 파악하기 어려운 문제를 해결하기 위해, 단순 질의응답을 넘어 진단·학습·복습·성장 추적 루프를 제공하는 AI 멘토 서비스입니다.

핵심 구현 및 아키텍처

- **3세션 구조** : 개념 학습 · 코드 리뷰 · 설계 면접으로 이어지는 진단 → 학습 → 복습 → 성장 추적 루프
- **역할별 LLM 에이전트 분리** : 대화 · 진단 · 평가 · 계획 · 회고 에이전트를 서비스 계층에서 순차 제어
- **LLM 응답을 스키마로 강제** : tool 정의로 형식을 고정하고 받은 값을 같은 스키마로 재검증
- Stack : Python · FastAPI · PostgreSQL(pgvector, RAG) · Redis · Claude API · Docker · Fly.io

Grit — 운동 기록 앱

개요 : 파편화된 PT 수업 및 개인 운동 기록을 통합, 온디바이스 자세 분석을 통해 사용자 경험을 개선한 앱입니다.

핵심 구현 및 아키텍처

- **운동 기록 통합** : PT 수업 기록 + 개인 운동 로그 + 팁 노트 검색 + 루틴 등록 + Streak 관리
- **모델 학습** : 공개 데이터셋 4종 10,242장을 한 클래스로 합쳐 YOLOv8s 학습
- **온디바이스 ML 추론** : 바벨 경로 추적 분석 — **Galaxy A36 기준 처리 시간 108초 → 20초 (81% 단축)**
- Stack : Kotlin · Spring Boot · PostgreSQL · React Native · Python(ML: YOLO, OpenCV)

주식 매매 엔진 — 주문 체결 코어

개요 : 주문 정합성과 처리 지연을 함께 다루는 주제를 골라 구축한 프로젝트입니다. 동일한 매매 플로우를 DB 락 기반 분산 처리와 인메모리 Single Writer 두 가지로 구현해 구조적 트레이드오프를 비교했습니다.

핵심 구현 및 아키텍처

- **v1 (RDB + Event-Driven) :** DB 비관적 락에서 출발해 Kafka 기반 이벤트 구조로 확장
- **v2 (In-Memory Engine) :** 링버퍼 단일 스레드 메모리 계좌 + UDP(Aeron) 통신, 주문 경로에서 브로커·DB 락 제거
- **성능 비교 측정** : 동일 주문 워크로드 (초당 192건) — **주문 발행 → 첫 체결 p99 지연 14.6ms → 1.6ms (약 89% 단축)**
- Stack : Java · Spring Boot · PostgreSQL · Kafka · Redis · LMAX Disruptor · Aeron(Archive) · JMH · k6

Aura — macOS 생산성 앱 (출시 준비 중)

개요 : 플로팅 형태의 할 일 앱과 AI와 연동되는 작업 환경, 그리고 보안 정보를 기록해두고 언제든 열어볼 공간이 필요해 만든 macOS 네이티브 앱입니다.

핵심 구현 및 아키텍처

- 할 일·메모·캘린더·Vault(보안 저장소) 기능 구성
- **내장 MCP 서버로 외부 AI 클라이언트가 할 일·메모를 직접 조작**
- 프로토타입을 먼저 만들고 https://github.com/devtaco30/shelf , Swift 로 Native 형태로 재구현
- Stack: Swift (macOS native) · Built-in MCP server

교육

[이스트캠프] 와썹 AI 모델 개발 | 2025.11 - 2026.04

- 백엔드 엔지니어로서 AI 모델 서빙 아키텍처 및 데이터 파이프라인 설계를 이해하기 위한 기술 시야 확장
- **[팀 프로젝트] MBTI 기반 맛집 추천 시스템**
    - 한국어 리뷰 형태소 분석·텍스트 벡터화로 감성 분류 모델 학습·선정
    - MBTI 키워드 매핑과 감성 점수를 결합한 추천 알고리즘 설계, 그룹 TOP3 합의 로직 구현
- **[팀 프로젝트] 트럼프 발언 기반 금융 불확실성 지수(TSUI) 산출 시스템**
    - 발언 약 22,000건과 섹터 ETF·VIX 데이터 수집 파이프라인 구축
    - NLP 감정 강도 분석과 시계열 누적 에너지 모델을 시장 변동성과 결합해 지수 산출 수식 설계
- **[팀 프로젝트] 독거노인 대상 AI 말동무 서비스**
    - 모델 학습을 제외한 전 구간 담당 — STT → LLM → TTS 파이프라인, 인터페이스·아키텍처 설계
    - 10~30초 걸리는 감정인식을 대화 흐름에서 분리해 비동기 처리, Redis에 적재해 다음 호출에 주입

[KCI] Java 웹개발자 양성 과정 | 2019.12 - 2020.06

- Java Backend 웹 개발 과정 학습
- Javascript와 Vue.js, React 등 Frontend 프레임워크 학습

학력

- 2017.06 - 2020.02 **학사** **| 국가평생교육진흥원** | 컴퓨터공학과
- 2006.03 - 2014.03 **학사(중퇴)** **| 홍익대학교(서울)** | 컴퓨터공학과

(주)웨이브릿지 2020.06 - 2023.08

01 **실시간 가상자산 지수 정합성 확보** 2
원인 추론부터 구조적 격리까지
ElasticJob · ZooKeeper

(주)블루밍비트 2023.08 - 2025.08

02 **이벤트 기반 구조 전환** 3
전달 처리와 뉴스 발행의 결합 해소
Kotlin · Kafka

03 **알림 발송 구조 전환** 4
설정 미적용으로 발생한 누락·오발송 해결
Kotlin · FCM

04 **검색 응답 지연 단축** 5
1.5~3s → 200~450ms
Kotlin · Coroutine · OpenSearch

Side Builds

05 **Apex AI 코치 응답 평가** 6
수정 전후 비교 기준 수립
Python · Claude API (Haiku 4.5)

06 **주식 매매 엔진** 7
주문 접수 경로에서 체결까지 **p99 14.6ms → 1.6ms**
Java · LMAX Disruptor · Aeron · Kafka · PostgreSQL

(주)웨이브릿지 | 2020.06 - 2023.08

실시간 가상자산 지수 정합성 확보 - 원인 추론부터 구조적 격리까지

[기존 구조의 한계] : 원인 미상의 지수 이상치

퀀트가 설계한 지수 로직을 실시간 시스템으로 구현하는 프로젝트를 담당했습니다. PM과 1:1로 붙어 백엔드와 프론트, 인프라를 함께 맡았습니다. 국내외 거래소 11곳에서 가상자산 오더북을 소켓으로 상시 수신해, 1초마다 마감해 지수를 산출하고 적재했습니다. 24/7 무중단으로 하루 100만 건 이상이 쌓였습니다. 별도의 모니터링 체계 없이 운영하던 중, 서비스 차트에서 **지수가 튀는 현상**을 발견했습니다. 지수는 차트와 대시보드 등 서비스 전반의 기반 데이터인 동시에, 사용자가 투자 판단을 내리는 근거였습니다. 수치가 틀리면 **사용자의 잘못된 투자 판단**으로 이어질 수 있는 리스크였습니다.

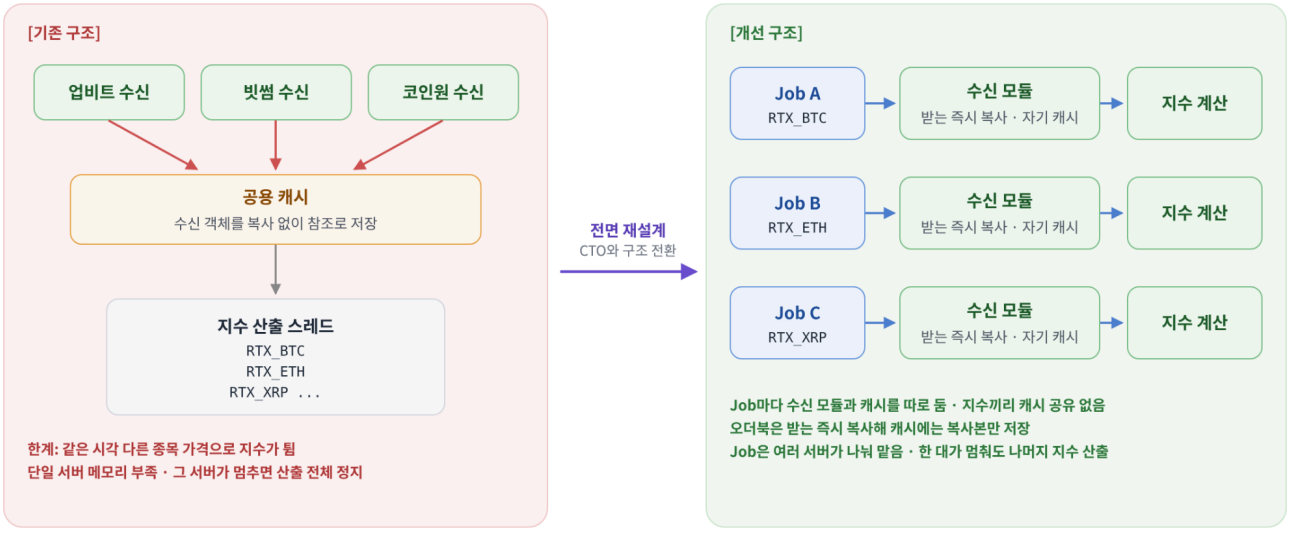
[판단과 해결] : 지수별 산출 작업과 캐시 분리

1단계 : 데이터 패턴 분석으로 공유 캐시 특정, 맵 교체와 종목 검사

스레드 상태를 직접 추적해봤지만 원인을 특정하기 어려웠고, 방향을 바꿔 튀는 수치 자체를 분석했습니다. 한 종목의 가격이 같은 시각, 가격대가 전혀 다른 종목의 가격 근처로 튀는 패턴이 보였습니다. **서로 다른 종목의 데이터가 섞인다는 가설**로 CTO와 논의했고, 데이터가 모이는 곳을 따라가 거래소별 수신 스레드 여럿이 쓰고 지수별 산출 스레드가 읽는 **공유 캐시**를 찾았습니다. 처음에는 동시 접근을 의심했지만, HashMap은 조회할 때 키를 비교하기 때문에 경합만으로는 다른 종목의 값이 나올 수 없다는 걸 확인했습니다. 다만 항목 **유실**은 생길 수 있어 캐시 맵을 ConcurrentHashMap으로 바꿨고, 섞인 오더북이 계산에 들어가지 않도록 **종목을 확인하는 검사**를 넣었습니다. 이후 값이 튀는 현상은 멎었지만, 섞이는 원인은 아직 찾지 못한 상태였습니다.

2단계 : 받는 즉시 복사, 지수별 독립 Job 으로 분리

이후 코드를 리뷰하다가, 캐시가 수신 라이브러리에서 받은 오더북 객체를 **복사 없이 참조로 저장**하고 있다는 것을 발견했습니다. 라이브러리가 그 객체를 다른 종목의 내용으로 다시 채우면, 캐시에 있는 값도 그대로 다른 종목의 호가가 되는 구조였습니다. 맵 교체로는 막을 수 없는 문제였습니다. 여기에 자산이 늘면서 **단일 서버의 메모리가 부족**해졌고, 그 서버가 멈추면 지수 산출 전체가 멈추는 문제도 겹쳤습니다. CTO와 구조 방향을 논의해 **지수별로 독립된 Job 단위로 분리**했고, 재설계와 구현을 맡았습니다. 오더북은 **받는 즉시 복사**해 캐시에는 복사본만 넣었습니다. 캐시도 Job마다 따로 두었습니다.



[성과] : 산출 작업 간 간섭 차단 및 확장성 확보

Job을 여러 서버가 나눠 맡는 구조가 되면서 **자산이 늘어도 서버를 늘려 감당**할 수 있게 됐고, 실행 주체가 여러 대라 한 대가 멈춰도 나머지 지수 산출은 계속됩니다. 지수 산출을 Job 단위로 나누면서, **새 지수는 DB에 지수 스펙을 등록하는 것**만으로 추가할 수 있게 됐습니다. 모니터링 부재가 발견을 늦춘 점을 반영해, 지수 변화량이 일정 기준을 넘으면 Slack 알림을 보내도록 했습니다. 이후 이 알림이 같은 원인으로 울린 적은 없습니다.

(주)블루밍비트 | 2023.08 - 2025.08

이벤트 기반 구조 전환 - 전달 처리와 뉴스 발행의 결합 해소

[기존 구조의 한계] : 뉴스 발행 코드에 쌓이는 전달 처리와 결합도 심화

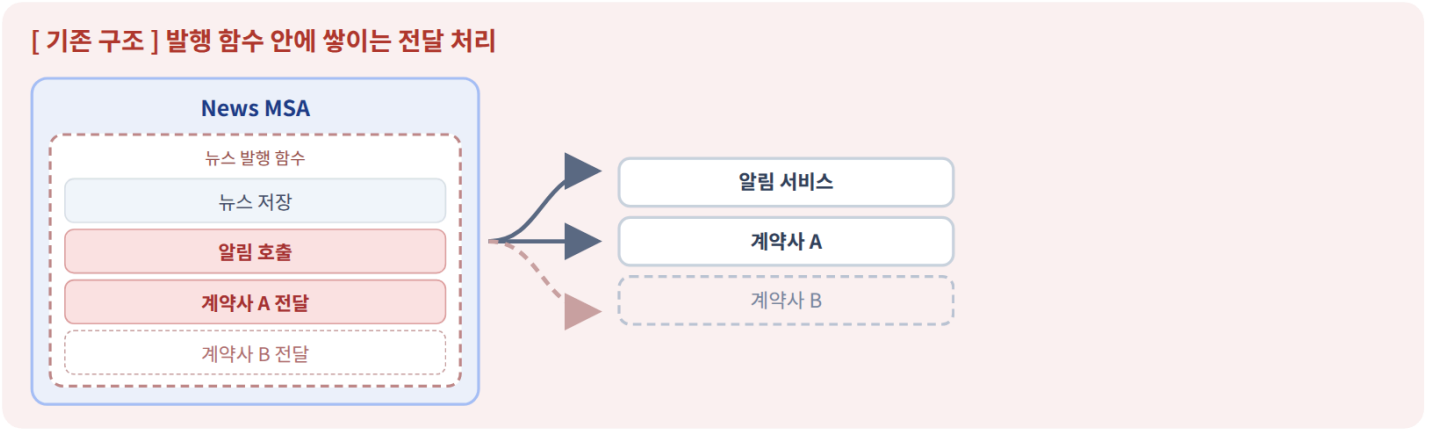
뉴스 발행 함수 안에는 알림 API 호출과 계약사A로 정제본을 보내는 FTP 업로드가 함께 들어 있었습니다. 정제본을 받는 계약사가 늘면서 전달 처리를 발행 코드에서 떼어내야 했습니다. 계약사나 후속 작업이 하나 늘 때마다 발행 코드를 고쳐야 했고, 알림 API 호출이나 FTP 업로드 중 하나가 실패하면 뉴스 발행 전체가 영향을 받는 구조였습니다.

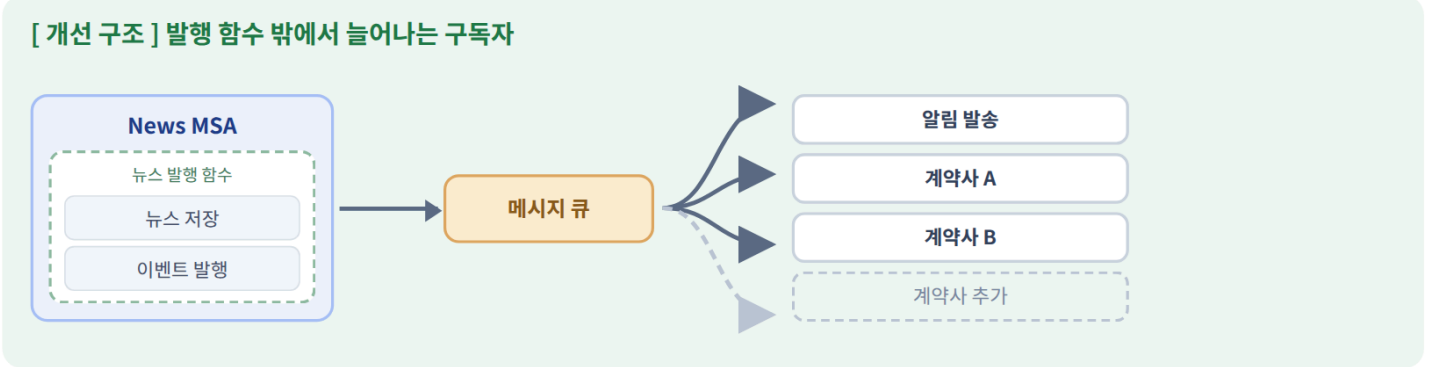
[판단과 해결] : 메시지 큐 도입 및 후속 작업 구독 전환

전달 처리를 별도 MSA로 빼는 것만으로는 부족했습니다. 뉴스 발행이 그 MSA를 직접 호출하면 호출 대상만 바뀔 뿐 결합은 그대로였습니다. 뉴스 발행 주체가 후속 작업의 성공 여부까지 책임지는 구조 자체가 잘못됐다고 판단했고, 이를 팀에 제안하여 메시지 큐 기반의 비동기 구조로 전환했습니다. 뉴스 발행은 큐에 이벤트를 올리면 끝나고, 각 후속 작업이 그 이벤트를 구독해 각자 처리하게 했습니다. 전환은 뉴스 MSA 담당자와 나눠 진행했습니다. 발행에서 이벤트를 내보내는 코드는 그 담당자가 맡았고, 이벤트를 받아 처리하는 MSA들은 제가 만들었습니다.

[성과] : 뉴스 발행과 후속 작업의 장애 분리

구조 분리 이후 후속 서비스의 장애가 뉴스 발행에 영향을 주지 않게 되었습니다. 신규 계약사 확장이나 알림 조건 추가 시에도 메인 로직을 건드리지 않고 대응할 수 있게 되었습니다. 요구사항이 자주 바뀌는 환경이었는데, 이후 계약사가 늘 때마다 FTP, RSS, API처럼 요구하는 전달 방식이 달라도 뉴스 발행 쪽은 그대로 두고 받는 쪽만 추가해 대응했습니다.





알림 발송 구조 전환 - 설정 미적용으로 발생한 누락·오발송 해결

[기존 구조의 한계] : 탐지 불가한 외부 SaaS 결함과 알림 신뢰도 저하

알림을 꺼도 알림이 온다는 내부 제보가 들어왔습니다. 앱 개발자, 백엔드 동료와 여러 기기로 테스트해 범위를 좁혔고, 로그아웃하지 않은 채 앱을 삭제했다가 다시 설치하면 **OneSignal이 사용자를 식별하지 못해** 알림 설정이 적용되지 않는다는 것을 확인했습니다. 그 결과 받아야 할 알림이 누락되거나 꺼 둔 알림이 발송됐습니다. 이 경우에도 OneSignal API는 성공 응답을 반환했고 발송 기록에도 성공으로 남았기 때문에, 우리 쪽에서는 감지할 방법이 없었습니다. 투자 관련 실시간 뉴스 서비스라 알림은 사용자를 다시 앱으로 불러오는 주요 경로였습니다.

[판단과 해결] : Firebase 전환 및 알림 설정 판정 내재화

알림 설정 판정만 백엔드로 가져오고 발송은 OneSignal에 계속 맡기는 방법도 있었습니다. 하지만 그러면 발송 결과는 여전히 같은 응답에 의존해야 해서, 비슷한 문제를 또 놓칠 수 있었습니다. 그래서 알림 설정 판정과 발송을 모두 백엔드에서 하도록, FCM으로 발송하게 전환했습니다. 토큰 발급과 저장은 앱 개발자와 앱푸시 담당자가 맡았고, 저는 발송 인터페이스를 맡아 FCM 응답을 그대로 발송 기록에 남기고 실패하면 Slack으로 알림이 오게 했습니다.

[성과] : 알림 누락 해소 및 발송 실패 확인

전환 이후에는 앱을 다시 설치해도 백엔드에 저장된 알림 설정이 그대로 적용됩니다. 발송 기록은 전에도 남겼지만 OneSignal이 준 값이라 믿을 수 없었고, FCM으로 바꾼 뒤로는 FCM 응답이 그대로 기록에 남아 발송 실패를 바로 확인할 수 있게 됐습니다.





검색 응답 지연 단축 - 1.5~3s → 200~450ms

[기존 구조의 한계] : 파편화된 검색 구현과 직렬 조회로 인한 병목

내부 사용자 제보로 검색 지연 문제를 인지했고, 게이트웨이에 프로메테우스를 붙여 병목 구간을 특정했습니다. 각 모듈마다 검색 구현 방식과 라이브러리가 달라 병목의 원인을 추적하기 어려운 구조였고, 1차 검색 이후 코인 정보·기자 정보·댓글 등을 직렬로 추가 조회하면서 지연이 누적됐습니다. 빠른 정보가 투자 판단으로 이어지는 서비스 특성상, 검색 타임아웃이 반복되면 사용자가 필요한 정보를 제때 찾지 못했습니다.

[판단과 해결] : 검색 책임 일원화 및 병렬 조회 전환

검색은 단순 조회와 달리 여러 조건을 조합해야 하는데, 뉴스·커뮤니티 MSA는 이미 맡은 책임이 많았습니다. 각 MSA의 검색 구현을 하나씩 손보는 방법도 있었지만, 구현이 제각각이라 도메인마다 전부 리뷰해야 했습니다. 그래서 검색을 Search MSA 하나로 모아 클라이언트가 호출하는 창구를 하나로 줄였습니다. 코인 정보 같은 추가 조회도 Search MSA가 맡게 했고, 병렬로 처리하되 요청이 몰릴 때 한 요청의 조회가 다른 요청까지 느리게 만들지 않도록 동시에 실행되는 조회 수는 제한했습니다.

[성과] : 간헐적 검색 오류 해소 및 조회 책임 분리

평소 1.5~3초, 심할 때는 10초 타임아웃까지 갔던 검색이 200~450ms로 줄었고, 간헐적인 검색 오류도 사라졌습니다. 검색 구현이 Search MSA 한곳에 모이면서 뉴스·커뮤니티 MSA가 검색 때문에 자기 책임이 아닌 데이터를 조회할 일도 없어졌습니다. 대신 서비스와 호출 단계가 하나씩 늘었습니다. 뉴스·커뮤니티 쪽 데이터 구조가 바뀌면 Search MSA도 함께 고쳐야 했습니다.





해빗웍스 | 2026.04 -

Apex — AI 코치 응답 평가 - 수정 전후 비교 기준 수립

[배경] : LLM 응답의 무작위성과 비교 기준의 부재

Apex의 AI 코치는 정답을 바로 알려주지 않고, 학습자가 개념을 자기 말로 설명하도록 질문을 던집니다. LLM은 같은 질문에도 매번 다르게 답합니다. 코칭 프롬프트를 수정하는 작업 뒤에는 응답이 나아졌는지 직접 세션을 진행해보며 확인했었습니다.
하지만 나아진 것이 **수정** 때문인지 **LLM 자체의 무작위성** 때문인지 알 수 없었습니다.

[설계] : LLM 응답 측정 기준과 재실행 비교 방식

수정을 하지 않은 상태에서 결과가 다르게 나오는 비율을 측정하고, 그 수치보다 높을 때만 효과로 인정하기로 했습니다. (LLM 자체의 변동폭을 넘어섰을 때, 유효하다는 판단)

- **합격선** : 수정한 지침이 테스트용 대본 실행에서 지켜지는지, 지켜진 비율이 얼마인지를 지정합니다.
- **비교 상대** : 기존 프롬프트로 한 번 더 실행한 결과로 비교합니다.(기존 결과값과 비교하면, 그 값을 기반으로 프롬프트를 수정한 것이기 때문에 수정본의 결과가 우위를 점하기 쉽습니다.)
- **과적합 확인** : 수정에 사용되지 않은 표현으로 대본을 새로 만들어 수정 전후 프롬프트로 다시 비교합니다.

[효과와 대가] : 블라인드 판정 통과와 턴당 호출 증가

‘**정답을 먼저 주지 않는다**’는 지침을 프롬프트에서 고쳤습니다. 수정에 사용되지 않은 주제로 대본을 새로 만들어 돌렸더니 지켜진 횟수가 **합격선**에 못 미쳤습니다. 그래서 이번엔 **코드 구조를 바꿨습니다**. 학습자가 연속으로 막힌 횟수를 코드에서 세어 코치에게 넘기고, 코치 응답에 정답이 섞였는지 검사해 섞였으면 응답을 다시 만들게 했습니다.



위 통과 수는 LLM이 판정한 값입니다. 어떤 버전에서 나온 응답인지 LLM이 알고 있는 상태에서 판정했습니다. 그래서 두 버전의 응답을 섞은 후, 어느 버전인지 모르는 채로 **사람이 직접 라벨**을 붙였습니다. 코드 구조를 바꾼 버전에서는 답이 노출됐다고 표시한 응답이 없었습니다.
이렇게 되면 대화 한 턴에 LLM을 최대 네 번 호출하게 됩니다. 응답 한 건에 약 $0.01이 사용됩니다(Haiku 4.5, 24건에 $0.24). 다시 만든 응답도 반려되면 **미리 적어 둔 질문**을 그대로 사용하면서, 프로덕트 지침인 ‘정답을 먼저 주지 않는다’를 유지하게 됩니다.

개인 프로젝트 | 2026.02 -

주식 매매 엔진 - 주문 접수 경로에서 체결까지 p99 14.6ms → 1.6ms

[배경] : DB 로우 락 점유와 트랜잭션 처리 지연

초기 구현은 주문 접수를 DB 트랜잭션으로 처리했습니다. 계좌 로우에 비관적 락을 걸고, 잔고·예약 증거금·미결제 미수금을 검증한 뒤 주문을 저장하는 구조입니다. 같은 계좌의 다음 주문은 이전 트랜잭션이 커밋될 때까지 기다렸고, 체결 뒤에도 보유 수량과 미수금을 반영하려고 같은 계좌 상태를 다시 갱신했습니다.
대기를 줄이려고 주문 처리 컨슈머와 Kafka 파티션을 3개에서 9개로 늘렸고, 처리량은 초당 227건에서 385건이 됐습니다. 다만 파티션을 늘려도 같은 계좌의 주문은 여전히 DB 락 안에서 하나씩 처리됐습니다.

[판단과 대안] : 메모리 상태와 단일 소비 스레드 도입 및 DB 락·브로커 제거

구조 재설계 시 지켜야 할 조건은 세 가지였습니다.

- 주문 접수와 체결 과정에서 DB 잠금, 트랜잭션 처리, 메시지 전달을 기다리는 시간을 없앨 것
- 여러 주문이 동시에 들어와도 같은 계좌의 잔고와 주문 상태가 입력 순서대로 처리되도록 할 것
- 기존 시스템을 유지한 채 동일한 주문을 신규 시스템에도 처리시켜 성능과 결과를 비교할 수 있도록 할 것

계좌 잔고와 주문별 증거금을 워커 **메모리**로 옮겼습니다. 주문·체결·정산 이벤트는 **링버퍼**로 받고, 계좌 상태는 **단일 소비 스레드만 고치게** 했습니다. 그래서 같은 계좌의 상태는 하나의 순서로 갱신되고, 락 없이도 동시 갱신 충돌이 생기지 않습니다. 매수 가능 금액도 DB 조회 없이 메모리의 값으로 계산합니다.

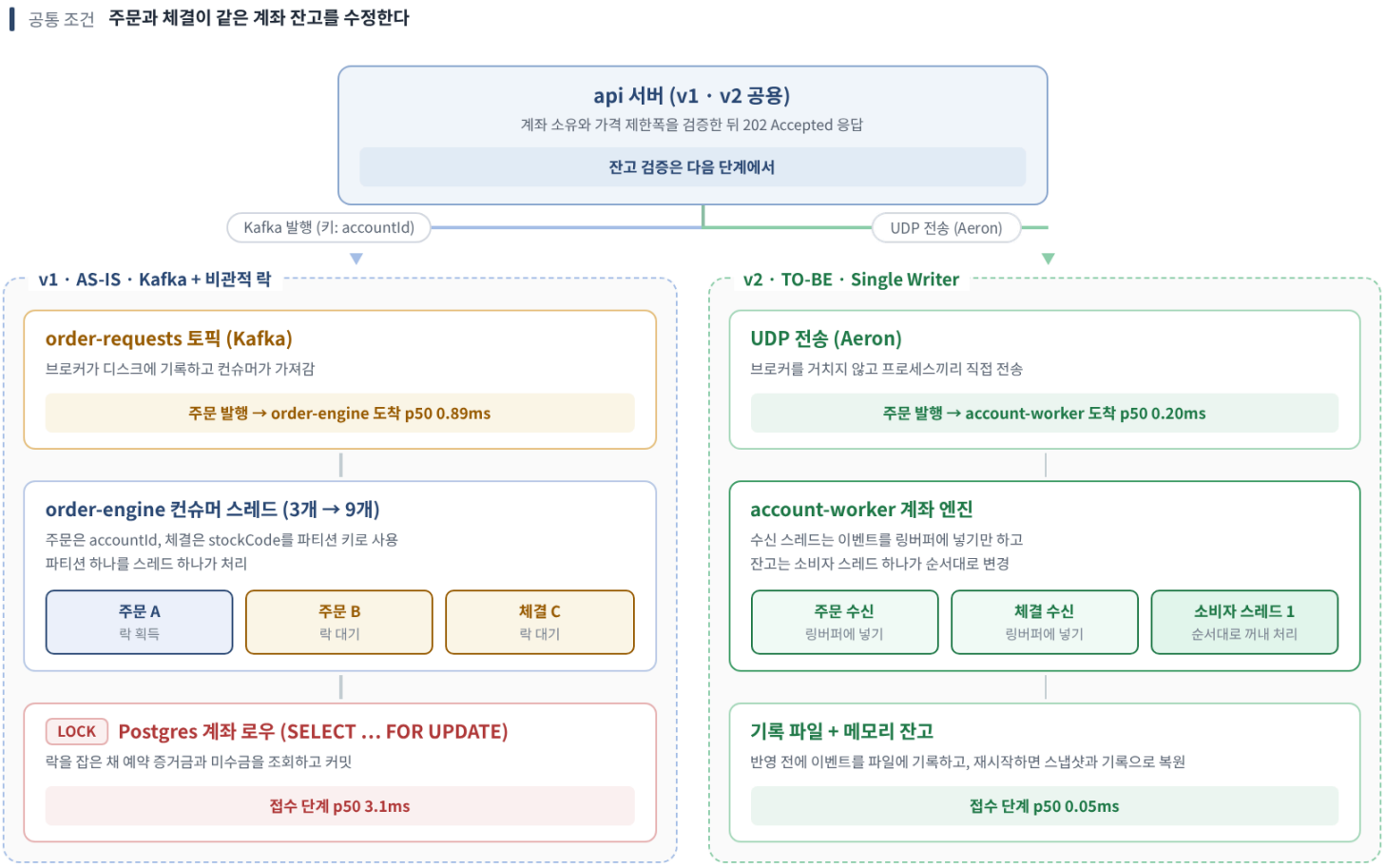
접수·체결 경로에서는 Kafka를 빼고 워커끼리 직접 통신하게 했습니다. 이벤트는 반영 전에 파일에 먼저 기록하고, 재시작하면 스냅샷과 그 뒤의 기록을 다시 읽어 메모리 상태를 복원합니다. 계좌 상태는 Kafka를 거쳐 조회용 DB에 묶어서 반영하고, 정산 기록은 DB에 저장합니다.

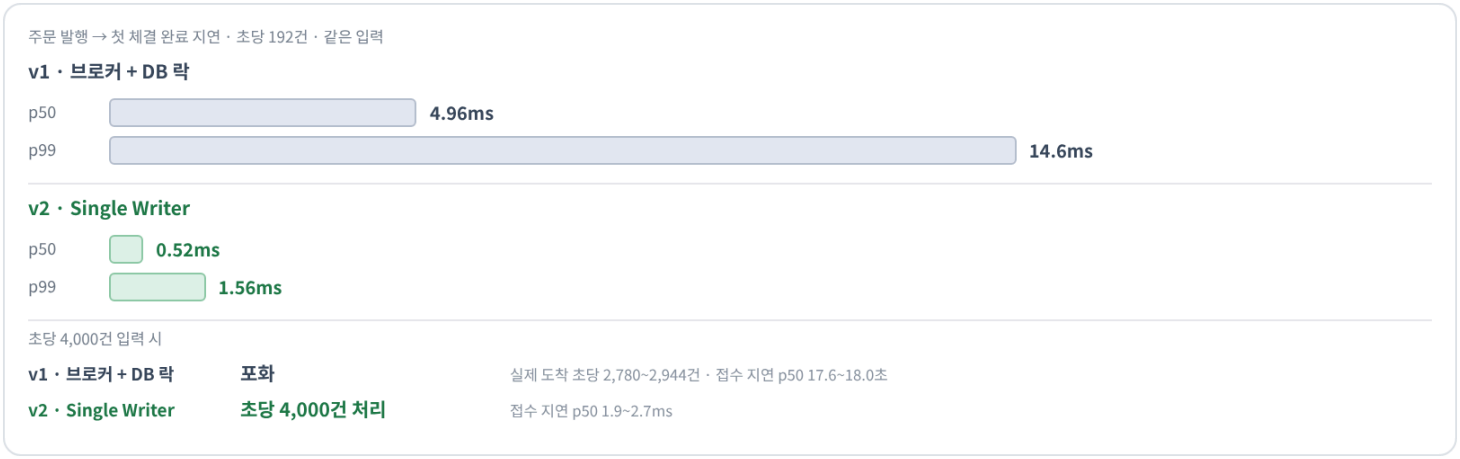
검토한 대안 :

- **락 안의 쿼리 최적화 :** 적용. 여러 로우를 조회해 합산하던 것을 DB SUM 한 번으로 바꿔 락 보유 시간은 줄었으나, 락을 잡은 채 DB를 왕복하는 구조는 그대로.
- **조건부 원자 UPDATE** : WHERE 절에 조건을 걸어 한 문장으로 검증과 예약을 끝내 앱이 락을 잡지 않게 하려 했으나, 매수 가능 금액이 잔고·예약증거금·미수금 **세 개의 테이블**에서 나와 **WHERE 절 하나에 담기지 않음.**
- **Redis Lua 스크립트:** 잔고와 예약을 Redis에 두고 스크립트 하나로 원자적으로 처리. DB 락은 피하지만 주문마다 네트워크 왕복이 남아 경로에서 네트워크를 빼려는 목적과 반대고, 내구성과 복구까지 Redis에 맡기게 됨.

[효과와 대가] : 체결까지의 지연 감소 및 중복 방지·복구의 직접 처리 부담

같은 입력을 초당 192건으로 넣었을 때, 주문 발행부터 첫 체결 완료까지 걸린 시간은 p50과 p99 모두 약 10분의 1로 줄었습니다. 가장 크게 줄어든 구간은 DB 락·조회·커밋을 거치던 접수 단계로, 약 60배 차이가 났습니다.





두 구현은 전달 방식부터 달라서, 조건을 맞추지 않으면 이 차이가 구조에서 나온 것인지 설정에서 나온 것인지 구분할 수 없습니다. 그래서 입력·도착 속도·로그 수준·응답 시점을 같게 두었고, 판정 기준은 측정 전에 문서로 정했습니다.

대신 DB가 해 주던 중복 방지와 복구를 애플리케이션이 직접 맡게 됐습니다. 같은 주문이나 체결이 두 번 들어와도 한 번만 반영되도록 직접 걸러야 합니다. 상태가 메모리에 있어서, 프로세스가 죽으면 기록을 다시 읽어 복원하는 동안 해당 계좌의 처리가 멈춥니다.